

How to use Socket.io with Next.js API Routes

Asked 3 years, 9 months ago Modified 10 months ago Viewed 39k times



47



Next.js provides [serverless API routes](#). By creating a file under `./pages/api` you can have your service running, and I want to have a Socket.io service by using this mechanism.

I have created a client:

`./pages/client.js`

```
import { useEffect } from 'react';
import io from 'socket.io-client';

export default () => {

  useEffect(() => {
    io('http://localhost:3000', { path: '/api/filename' });
  }, []);

  return <h1>Socket.io</h1>;
}
```

And an API route:

`./pages/api/filename.js`

```
const io = require('socket.io')({ path: '/api/filename' });

io.onconnection = () => {
  console.log('onconnection');
}

io.on('connect', () => {
  console.log('connect');
})

io.on('connection', () => {
  console.log('connection');
})

export default (req, res) => {
  console.log('endpoint');
}
```

But I can't get the client to connect to the Socket.io server and successfully see any of:

`'onconnection'`, `'connect'`, or `'connection'` printed.

`socket.io`

`next.js`

[Share](#) [Improve this question](#) [Follow](#)

asked Aug 15, 2019 at 15:45



Aurelius

679 ● 1 ● 6 ● 16

2 Seems like not supported as of yet : github.com/zeit/next.js/issues/8311 – [eenagy](#) Aug 18, 2019 at 7:59

3 Answers

Sorted by:

Highest score (default)

The trick is to plug 'socket.io' into the http server only once, so checking every access to the api.
Try something like this:

52

`./pages/api/socketio.js`

```
import { Server } from 'socket.io'

const ioHandler = (req, res) => {
  if (!res.socket.server.io) {
    console.log('*First use, starting socket.io')

    const io = new Server(res.socket.server)

    io.on('connection', socket => {
      socket.broadcast.emit('a user connected')
      socket.on('hello', msg => {
        socket.emit('hello', 'world!')
      })
    })

    res.socket.server.io = io
  } else {
    console.log('socket.io already running')
  }
  res.end()
}

export const config = {
  api: {
    bodyParser: false
  }
}

export default ioHandler
```

`./pages/socketio.jsx`

```
import { useEffect } from 'react'
import io from 'socket.io-client'

export default () => {
```

```

useEffect(() => {
  fetch('/api/socketio').finally(() => {
    const socket = io()

    socket.on('connect', () => {
      console.log('connect')
      socket.emit('hello')
    })

    socket.on('hello', data => {
      console.log('hello', data)
    })

    socket.on('a user connected', () => {
      console.log('a user connected')
    })

    socket.on('disconnect', () => {
      console.log('disconnect')
    })
  })
}, []) // Added [] as useEffect filter so it will be executed only once, when
component is mounted

return <h1>Socket.io</h1>
}

```

Share Improve this answer Follow

edited Jun 1, 2021 at 17:29

answered Jun 24, 2020 at 3:29



Ajith Gopi

1,489 ● 1 ● 12 ● 20



rogeriojlle

1,046 ● 1 ● 11 ● 19

- 1 How can this be used with external applications? Say I have a mobile app that's connecting to this API and want to listen for document change inside models. For example, Mongoose `post('save')` . – [francis](#) Jul 11, 2020 at 15:10
- 9 One problem with this solution in development is that after `res.socket.server.io` is set, you can't take advantage of hot-reloading for code inside that if statement. – [Daniel](#) Jul 25, 2020 at 5:34
- 11 I actually just switched everything to Pusher because Vercel says in their [limitations docs](#) that websockets are not supported in serverless functions. Also, they have a [guide](#) for how to set up pusher. – [Alex Cory](#) Sep 9, 2020 at 15:35
- 3 Yes pusher does use websockets, however when interacting with pusher you are only making REST API calls. POST to be specific. All the websocket stuff is handled clientside between your app and pusher. – [Alex Cory](#) Sep 14, 2020 at 21:48
- 2 1. you setup a serverless function that authenticates with pusher `/api/pusher/auth` 2. then your frontend uses the key from that to authenticate your frontend with pusher 3. I used [use-pusher](#) react hooks for listening for events on the frontend. Specifically `useEvent` and the `Provider` 4. I created endpoints such as a create-message endpoint, then after creating the message + saving to db, I used `pusher.trigger('private-channel-name', 'message:created', message)` – [Alex Cory](#) Sep 14, 2020 at 22:45



You have to have the `/api/pusher/auth` to authenticate with pusher on the frontend. Then you use the key you get from that to communicate with pusher. It's for security purposes. You can do it all through the frontend, but depending on your app, if you're saving data (such as messages, or chats) then probably should authenticate.



Share Improve this answer Follow

answered Sep 17, 2020 at 0:12



Alex Cory

10.4k ● 10 ● 51 ● 62

1 So is this how we authenticate: `var pusher = new Pusher({ appId: '', key: '', secret: '', cluster: 'eu', encrypted: true });` – [traderjosh](#) Sep 22, 2020 at 17:36

Hopefully [this gist](#) helps. – [Alex Cory](#) Sep 23, 2020 at 0:19

Thanks for the response - I got up to that point, but I had some issues with [use-pusher](#), particularly the `usePresenceChannel` because I wanted to track my users and display them in the chatroom, but presence channel doesn't work sadly. Have you tried that? It was throwing some warnings for no auth headers with the `presenceChannel` and not registering the trigger events. I didn't find this problem with the normal `channel` like your gist. – [traderjosh](#) Sep 23, 2020 at 20:16

Did you post an issue in the repo? That's a question for the use-pusher author. – [Alex Cory](#) Sep 23, 2020 at 21:05

That's a good idea - I just did here: github.com/mayteio/use-pusher/issues/26. I was thinking of writing writing my own implementation to avoid these kind of issues before you recommended the library. It seems well-written so maybe I'm doing something wrong. Have you tried the presence channel? How else would you track your chatroom users? – [traderjosh](#) Sep 25, 2020 at 20:01



You can use custom server and attach sockets to it (just like with express) and provide needed path where socket.io will listen. [How to use custom server](#)

You can write something like this `server.js`

```
const { createServer } = require('http');
const { parse } = require('url');
const next = require('next');
const { Server } = require('socket.io');

const dev = process.env.NODE_ENV !== 'production';
const hostname = 'localhost';
const port = 3000;
// when using middleware `hostname` and `port` must be provided below
const app = next({ dev, hostname, port });
const handle = app.getRequestHandler();

app.prepare().then(() => {
  const server = createServer(async (req, res) => {
    try {
      // Be sure to pass `true` as the second argument to `url.parse`.
      // This tells it to parse the query portion of the URL.
      const parsedUrl = parse(req.url, true);
```

```
const { pathname, query } = parsedUrl;

if (pathname === '/a') {
  await app.render(req, res, '/a', query);
} else if (pathname === '/b') {
  await app.render(req, res, '/b', query);
} else {
  await handle(req, res, parsedUrl);
}
} catch (err) {
  console.error('Error occurred handling', req.url, err);
  res.statusCode = 500;
  res.end('internal server error');
}
});

const io = new Server(server, {
  path: '/socket.io' // or any other path you need
});
```

You would need to run your server using `node server.js`

[Share](#) [Improve this answer](#) [Follow](#)

answered Jul 13, 2022 at 15:58



[Vitalii Yarmus](#)

17 ● 2